Workshop 2 — Design Artifacts and System Modeling

Course: Software Engineering II · **Semester:** 2026-II **Project:** Maulwurf — intelligent study assistant built from class recordings **Language:** English, as required by the workshop brief

Purpose

This folder contains the design deliverables for Workshop No. 2. It translates Maulwurf's functional specification and current architecture into artifacts that can guide implementation: system domain and actors, CRC cards, UML class diagrams, the relational database model, mockups, business process diagrams and a software architecture diagram.

The artifacts are intentionally honest about implementation status. A grey `planned` entity or a paragraph marked planned is part of the accepted roadmap, not a claim that the corresponding code already exists.

Deliverables

Workshop requirement	Deliverable
System domain, actor characterization and business rules	01-system-domain.md
CRC cards	02-crc-cards.md
UML class diagram with attributes, methods and relationships	03-class-diagram.md
Relational database model with tables, keys and relationships	04-relational-model.md
Mockups / wireframes	05-mockups.md
Business processes using activity diagrams	06-business-processes.md
Architecture diagram with components, layers and communication flows	07-architecture.md
Single submission artifact	Workshop-2.pdf

Source of truth and traceability

The workshop artifacts are derived from the repository documents and decisions:

- Functional and Technical Specification
- Database and API Design
- Architecture
- Processes
- ADR-0007 — Vite + React web stack
- ADR-0008 — API□ingest reception contract
- Claims registry

Build and review

The checked-in `.svg` files are generated from the `.puml` sources. The submission PDF is generated from the Markdown sections and the SVGs:

```
export PLANTUML_JAR=/path/to/plantuml-mit-1.2026.8.jar
export CHROME_PATH=/path/to/chrome-or-chromium
python3 Workshop-2/build.py
```

Build one stage only:

```
python3 Workshop-2/build.py diagrams
python3 Workshop-2/build.py pdf
```

`build.py` uses only Python's standard library, Java for PlantUML and headless Chrome/Chromium for PDF printing. The PlantUML MIT jar is verified by SHA-256 before diagram generation. No external provider, database, audio file, credential or private transcript is needed to build the workshop.

The PDF is the required single artifact. The Markdown, PlantUML and SVG files remain beside it so reviewers can inspect and reproduce every diagram.

1. System Domain

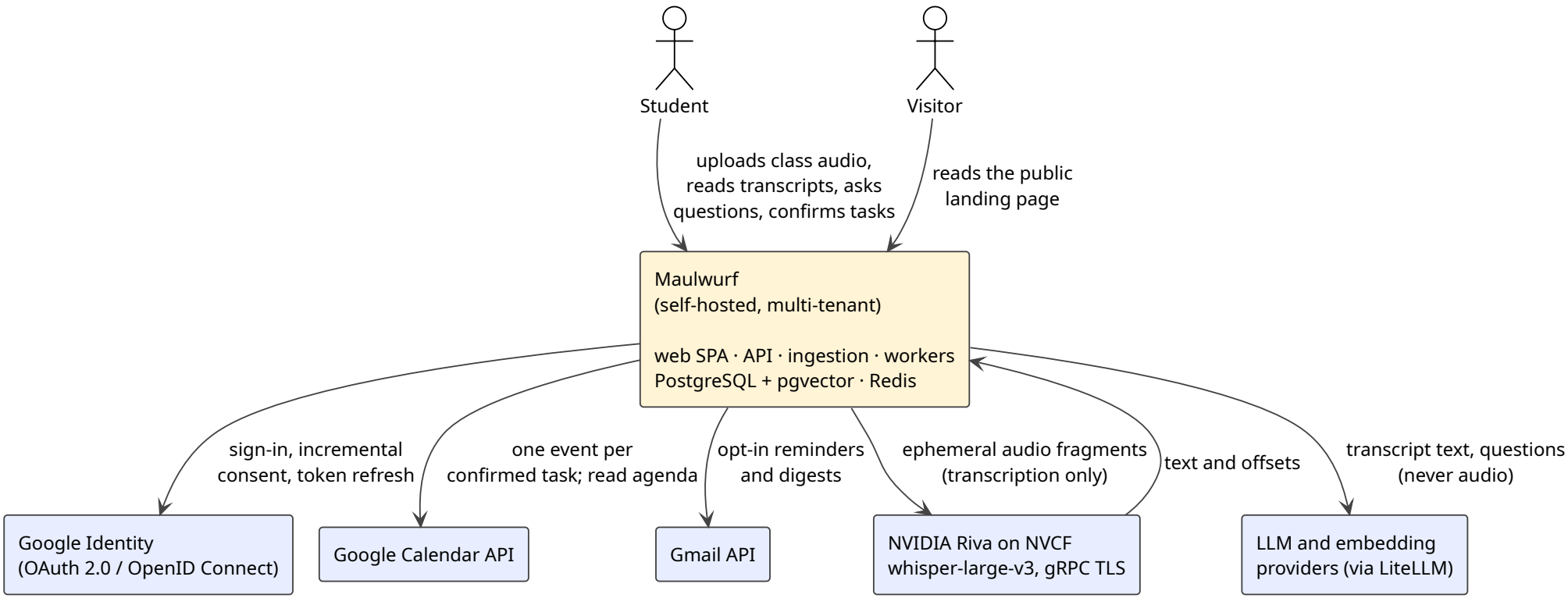
1.1 System context

Students record their classes and accumulate hours of audio they rarely listen to again. Exam dates, deadlines, readings and the teacher's warnings stay buried in that audio. Lookup tools such as NotebookLM let the student chat with a document, but they do not turn what the teacher said into scheduled, reviewable actions.

Maulwurf is a web study assistant that closes that gap. The student uploads a class recording; the system keeps only its durable **text**:

1. it transcribes the audio in the language the student selects, through NVIDIA Riva; 2. it deletes the audio and verifies the deletion before any text processing starts; 3. it indexes the transcript for hybrid search and for a chat that answers **with citations** to the exact transcript segment; 4. it proposes tasks, exams and readings extracted from the transcript, which the student reviews and, only after explicit confirmation, writes to Google Calendar; 5. it sends opt-in reminders and digests through the student's own Gmail account.

Figure 1.1 · System context of Maulwurf



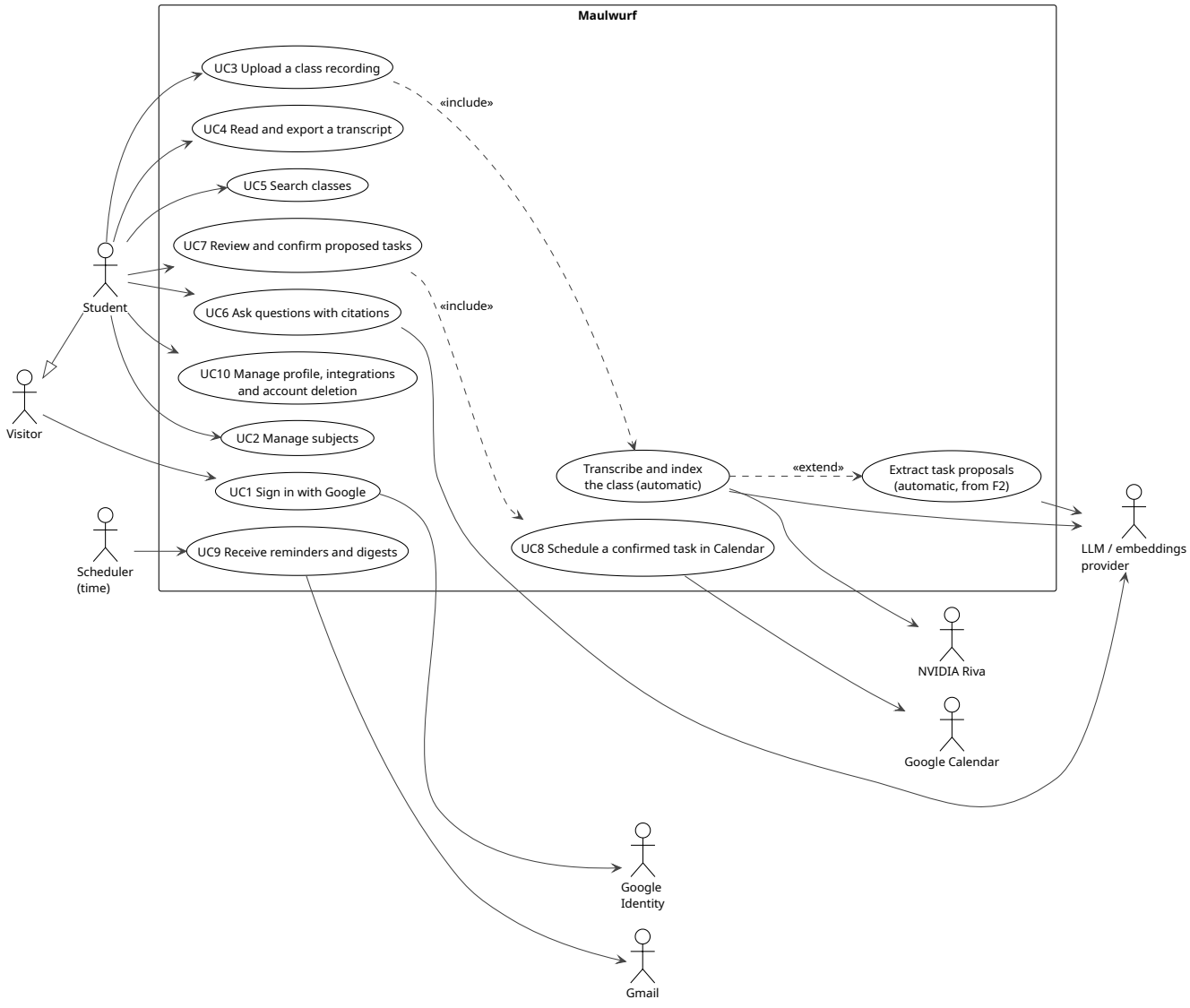
Boundary. Everything inside the box is self-hosted and multi-tenant: a React single-page application, a FastAPI API, an isolated ingestion service, ARQ workers and a scheduler, PostgreSQL with pgvector, and Redis, behind a Caddy reverse proxy. Everything outside is a third-party service reached through an adapter. Two flows cross the boundary with private data and are therefore gated by the business rules below: **audio** goes only to NVIDIA Riva, as ephemeral fragments, and **text** goes only to the LLM and embedding providers.

Scope. In scope for the MVP and v1: upload and transcription, structured extraction, Calendar events after confirmation, hybrid search, RAG chat filtered by subject, reading the agenda from chat, and email reminders and digests. Out of scope: diarization, video or link ingestion, collaboration, audio playback or download, automatic language detection, translation, a fully local mode, and native mobile apps (an installable PWA comes later).

Delivery phases. F0 foundation and provider spike; F1 upload → transcript → index → chat; F2 extraction → review → Calendar; F3 reminders, digests and close-out. They map to sprints S1–S4 of PLAN_SPRINTS.md.

1.2 Actor characterization

Figure 1.2 · Actors and main use cases



Actor	Type	Description	Goals	Interactions and permissions
Student	Primary, human	University student who records lectures (phone or laptop) in Spanish, English or French; uses a desktop or mobile browser; moderate technical skill; works alone (no shared spaces)	Not miss deadlines; find what was said in class; study from evidence	Signs in with Google; owns subjects, classes, transcripts, conversations and tasks; sees **only** their own data; is the only one who can confirm a Calendar write
Visitor	Primary, human	Anyone without a session	Understand the product and sign in	Public landing page and sign-in only; no private data
Scheduler	Internal, time	ARQ cron process with a single active role	Trigger reminders, digests and Calendar syncs on time	Acts on behalf of each tenant through the outbox; never reads audio
Google Identity	Secondary, external system	Google OAuth 2.0 / OpenID Connect	Authenticate the student by `sub`; grant incremental scopes	Called only by the API (state, PKCE, nonce); tokens stored encrypted
NVIDIA Riva (NVCF)	Secondary, external system	Hosted `whisper-large-v3` over gRPC TLS	Transcribe audio in the selected language	Receives audio fragments only from the ingestion service; transcription only, never translation; retention still unverified (decision D3-Audio)
LLM and embedding providers	Secondary, external system	Provisional `gpt-4o-mini` and `text-embedding-3-small` via LiteLLM	Embed chunks, answer chat questions, extract proposals	Receive text and questions, never audio or credentials; outputs are untrusted data validated against the database
Google Calendar API	Secondary, external system	The student's Google Calendar	Hold one event per confirmed task; provide the agenda	Written only after confirmation, through an idempotent job; read for the dashboard and chat tools
Gmail API	Secondary, external system	The student's own Gmail account	Deliver reminders and digests	Opt-in only; recipient is always the account owner
Operator	Supporting, human	Team member who deploys and monitors	Keep the service healthy and private	Infrastructure access; logs and metrics carry IDs only, never audio, transcripts or tokens
Teacher and classmates	Affected parties, not users	Voices in the recording	—	Never log in; the student acknowledges third-party voices before uploading

1.3 Business rules

Each rule cites its normative source in the specification (ESPECIFICACION.md) or the database and API design (DISENO_BD_API_SPRINTS.md).

ID	Rule	Source
BR-01	**No durable audio.** Audio exists only in bounded RAM (`tmpfs`) while it is received, validated, converted and transcribed: never on disk, object storage, Redis, logs, traces, caches or backups. There is no playback or download.	Spec §1 binding decisions 1–2
BR-02	**Deletion before text work.** Audio deletion starts when transcription ends or aborts; an attempt reaches `succeeded` only after the transcript is committed **and** cleanup is verified. Index and analysis jobs stay disabled until then.	Spec §1, §M3; Design §2.6
BR-03	**Lost audio means re-upload.** Without durable audio there is no server-side retranscription; analysis and indexing are retried from the stored text.	Spec §1 decision 5
BR-04	**Mandatory language.** The student selects the audio language from a published allowlist (`es`, `en`, `fr` today); no preselection, no autodetection, no `multi`, and the provider is never asked to translate.	Spec §1 decisions 3–4
BR-05	**Consent before bytes.** Uploading requires the current privacy-notice version, acceptance of cloud processing and acknowledgement of third-party voices; a missing or outdated version is rejected before capacity is reserved.	Spec §M2
BR-06	**Published limits.** Upload size and duration limits come from `/ingestion/capabilities` (target 200 MiB / 3 h, provisional until capacity decision D4); size is enforced while streaming and duration is measured with ffprobe.	Spec §M2
BR-07	**Deduplication by identity.** Identity = SHA-256 + language + subject + class date + timezone. An exact duplicate returns the existing class; the same audio with a different context requires an explicit, single-use, short-lived variant confirmation.	Spec §M2
BR-08	**Honest outcomes.** Silence or no usable text ends `rejected`; a technical loss ends `requires_reupload`; no empty transcript is padded or invented.	Spec §M2, §M3
BR-09	**Real timestamps only.** Precision is stored as `word`, `segment` or `none`; times are never invented by the LLM or interpolated; SRT export exists only with valid times.	Spec §M3
BR-10	**Tenant isolation.** Every business row belongs to one user; references use composite keys `(user_id, id)`; the user always comes from the server-side session, never from the request body or an LLM.	Spec §M1, §6; Design §2.1
BR-11	**One session authority.** The API issues an opaque `HttpOnly` cookie and requires CSRF token and same-origin `Origin` on every mutation; the ingestion service never sees the session, only a per-attempt internal credential.	Spec §M1; ADR-0008
BR-12	**Validated citations.** Chat answers cite only segments whose IDs were validated in the database for that tenant and version; without enough evidence the assistant says so.	Spec §M7
BR-13	**Human in the loop.** No Calendar write without explicit confirmation; recommendations never go straight to Calendar; chat tools are read-only.	Spec §1 principles, §M4, §M7
BR-14	**Dates against the class.** Dates are resolved from the class date and its timezone snapshot; `due_date` and `due_at` are mutually exclusive; ambiguous or missing dates go to review; `confidence_score` never authorizes auto-scheduling.	Spec §M4

ID	Rule	Source
BR-15	**Versioned edits.** Editing a confirmed task creates a pending revision; confirming needs the expected version, and a stale one fails with `409 version_conflict`. Re-analysis never overwrites edited, confirmed or scheduled tasks.	Spec §M4
BR-16	**Opt-in reminders.** Reminders at T-48 h, T-24 h and T-2 h for timed tasks, T-3 d and T-1 d at local time for all-day tasks, plus daily and weekly digests; a delivery whose outcome is unknown is never resent automatically.	Spec §M8
BR-17	**Deletion purges derivatives.** Deleting a class purges its transcript, chunks, embeddings, derived tasks and citations; existing Google events are kept unless the student explicitly asks to delete them. A subject with active classes is not deleted without explicit confirmation.	Spec §M2
BR-18	**Replaceable providers, versioned indexes.** Providers sit behind adapters; changing the embedding model or dimension creates a new index generation and a full reindex, never mixed vectors.	Spec §1 principles, §M6

2. CRC Cards

A CRC card (Class, Responsibilities, Collaborators) lists what a class knows and does, and which classes it needs to do it. The cards cover the **main classes** of the design: the persistent domain entities (left part of the class diagram) and the services that own the business processes. Each card states its module and how much of it exists in the repository today:

- **implemented**: the class exists on `develop` with tests;
- **partial**: the class exists, but part of its behavior is still a planned ticket;
- **planned**: designed in the specification, not implemented yet (ticket in brackets).

Class names follow the code and the specification. Some persisted fields keep the Spanish identifiers frozen in the extraction contract (`tipo`, `titulo`, `detalle`, `cita_textual`: type, title, detail and literal quote).

Domain entities

CRC-01 · User

Entity · `apps/api/app/models/core.py` · implemented

Responsibilities	Collaborators
Identify the student by the Google `sub`, never by email	Google Identity (via the auth service)
Keep profile data: email, display name, IANA timezone	—
Own every tenant resource (subjects, classes, conversations, tasks)	Subject, Audio, Conversation, Task
Track the account lifecycle `active` → `deleting` → `deleted`	Session, OutboxEvent (`account_deleted`)

CRC-02 · Session

Entity · `apps/api/app/models/core.py`, `services/sessions.py` · implemented

Responsibilities	Collaborators
Represent one opaque sign-in; store only the hash of its identifier	User
Hold the hash of the CSRF token shared by the session's tabs	—
Expire, rotate and be revoked (logout, account deletion)	User
Be the single source of the current user for every request	API dependencies (`core/deps.py`)

CRC-03 · Subject

Entity · `apps/api/app/models/core.py`, `routers/subjects.py` · implemented

Responsibilities	Collaborators
Group the classes of one course (name, color, teacher)	Audio
Keep names unique among the student's active subjects	User
Refuse deletion while it has active classes, unless explicitly forced	Audio

CRC-04 · Audio (class record)

Entity · `apps/api/app/models/ingestion.py` · partial (state and deletion: A2.6, A2.7)

Responsibilities	Collaborators
Describe one class without storing its audio: subject, class date, timezone, language, title	Subject
Hold the content identity (SHA-256 + context) used for deduplication	UploadService
Point to the active transcript, analysis and index versions	Transcript, Chunk
Derive the aggregate status `processing / ready / partial / failed` from its stages	IngestionAttempt, ProcessingRun
Act as tombstone when the class is deleted, so late jobs cannot write	OutboxEvent

CRC-05 · IngestionAttempt

Entity · `apps/api/app/models/ingestion.py`, `apps/ingest/maulwurf_ingest/lease.py` · implemented

Responsibilities	Collaborators
Record versioned consent before any byte is accepted	Audio, UploadService
Reserve ingestion capacity and expire the upload window	UploadService
Follow the ASR state machine `awaiting_upload` → `receiving` → `transcribing` → `transcript_committed_cleanup_pending` → `succeeded` (or `rejected`, `requires_reupload`, `cancelled`)	IngestionSupervisor
Carry the lease and fencing token so an expired owner cannot publish	IngestionSupervisor
Record cleanup status and evidence separately from the ASR result	IngestionSupervisor
Report real progress (bytes received, fragments done/total)	Web client (progress SSE)

CRC-06 · Transcript and Segment

Entity · `apps/api/app/models/transcript.py` · partial (schema implemented; commit: S2.5, A2.10)

Responsibilities	Collaborators
Store the full text of one version, with language, model and configuration	Audio
Declare the real timestamp precision `word / segment / none`	NvidiaRivaTranscriptionService
Keep ordered segments with Unicode character offsets and, only if valid, times	Segment
Be the evidence base for citations and extracted tasks	Chunk, Task, Message
Export TXT always and SRT only with valid times	Web client

CRC-07 · Chunk (with Embedding and IndexGeneration)

Entity · `apps/api/app/models/chunk.py` · partial (schema implemented; indexing: L2.1, L2.2)

Responsibilities	Collaborators
Split a transcript into ~800-token pieces with ~100 overlap, keeping character offsets	Transcript
Link each chunk to the segments it covers, which remain the unit of evidence	Segment
Hold one embedding per index generation (model, dimension, vector) and a full-text vector	Embedding, IndexGeneration
Publish a new generation atomically; never mix models or dimensions	IndexGeneration

CRC-08 · Conversation and Message

Entity · `apps/api/app/models/chat.py` · partial (schema implemented; endpoints: A2.12)

Responsibilities	Collaborators
Scope a chat to all classes, one subject or one class of the same student	Subject, Audio
Keep the ordered history with roles and statuses (`streaming`, `completed`, `failed`, `cancelled`)	Message
Make sending idempotent through `client_message_id`	RAGService
Record every source delivered to the model, cited or not, so deletions can purge answers	MessageSource, Chunk, Segment

CRC-09 · Task

Entity · planned (S3: A3.x)

Responsibilities	Collaborators
Represent one exam, assignment, deliverable, reading, reminder, recommendation or project	Audio, Subject
Keep the due date (`due_date`) or instant (`due_at`), never both, with its timezone	—
Move through `needs_review` / `pending` → `confirmed` → `completed`, or `dismissed`	TaskRevision
Hold the textual evidence (segment spans and literal quote) that justifies it	TaskEvidence, Segment
Track the Calendar sync status and the optimistic-locking version	GoogleCalendarService, OutboxEvent

CRC-10 · TaskRevision

Entity · planned (S3: A3.x)

Responsibilities	Collaborators
Hold proposed field changes for a confirmed task without touching its active fields	Task
Record the base version it was proposed against	Task
Become active only on confirmation with the current version; otherwise `409 version_conflict`	Task, OutboxEvent

Responsibilities	Collaborators
Carry re-analysis results as revisions, never as overwrites	AnalysisJob

CRC-11 · OutboxEvent

Entity · `apps/api/app/models/ingestion.py`, `workers/dispatcher.py` · partial (consumer acknowledgement and retries: J2.1)

Responsibilities	Collaborators
Record durable work in the same transaction as the change that causes it	Transcript, Task, User
Carry only IDs and non-sensitive configuration, never audio or tokens	—
Keep `index_requested` and `analyze_requested` blocked until cleanup is verified	IngestionAttempt
Be dispatched once per deduplication key, with bounded retries	OutboxDispatcher, Redis

Services

CRC-12 · UploadService

Service · `apps/api/app/services/uploads.py`, `services/audios.py` · partial (reception sink to ingestion: A2.13, S2.11)

Responsibilities	Collaborators
Validate metadata, language and consent, then reserve an attempt (`POST /audios`)	IngestionAttempt, Subject
Stream the binary `PUT` with incremental SHA-256 and cut it at the size limit	ReceptionSink
Decide exact duplicate, variant (with confirmation token) or new content	Audio, VariantConfirmationToken
Forward new content to the ingestion service over the internal network (ADR-0008)	IngestionSupervisor
Map every failure to a cataloged error code	—

CRC-13 · IngestionSupervisor

Service · `apps/ingest/maulwurf_ingest/supervisor.py` · partial (not yet reachable from the API: S2.11)

Responsibilities	Collaborators
Take the attempt's lease and keep it alive with a heartbeat	IngestionAttempt
Receive the stream into a private, bounded `tmpfs` directory	Receiver
Validate with ffprobe and normalize with ffmpeg, with no shell, network or unbounded resources	ffmpeg/ffprobe
Run ASR fragment by fragment in a killable subprocess with backpressure	NvidiaRivaTranscriptionService
Commit transcript and segments, then delete and **verify** the deletion in `finally`	Transcript, Cleanup
Close admission when cleanup fails and raise an alert	Admission

CRC-14 · NvidiaRivaTranscriptionService

Service (adapter of `TranscriptionService`) · `apps/ingest/maulwurf_ingest/asr/riva_client.py` · partial (not wired into the pipeline: S2.10)

Responsibilities	Collaborators
Send one normalized WAV fragment to Riva over gRPC TLS with the selected language	NVIDIA Riva
Never request translation or `multi`; reject codes outside the allowlist	—
Retry transient errors (at most 3, backoff with jitter) within the deadline; never retry auth, language or format errors	—
Support cancellation of the call in flight	IngestionSupervisor
Return words and offsets only when the provider really delivers them	Transcript

CRC-15 · SearchService

Service · planned (S2: L2.3, L2.4)

Responsibilities	Collaborators
Run a full-text branch (per-language `tsvector`) and a vector branch (pgvector, cosine)	Chunk, Embedding
Filter both branches by tenant, active version and tombstones **before** the top-k	IndexGeneration, Audio
Fuse results with Reciprocal Rank Fusion, deterministically	—
Serve `GET /search` and the chat retrieval	RAGService

CRC-16 · RAGService

Service · planned (S2: L2.5, L2.6)

Responsibilities	Collaborators
Build the context: question embedding, hybrid candidates, about eight chunks within the token budget	SearchService
Stream the answer over SSE (`delta`, `citation`, `done`, `error`)	Conversation, Message
Emit citations only to segment IDs validated in the database	Segment, MessageSource
Abstain when the evidence is insufficient	LLM adapter
From F2, call read-only tools (agenda, pending tasks) and propose tasks for review	GoogleCalendarService, Task

CRC-17 · AnalysisJob (`analyze(audio_id)`)

Service (ARQ job) · stub in `apps/api/app/workers/arq_app.py` · planned (S3: L3.x)

Responsibilities	Collaborators
Send the transcript in windows, with subject, teacher, class date and timezone	Transcript, LLM adapter
Validate the structured output with Pydantic and deterministic rules (dates, enums, spans)	`AnalysisResult` schema

Responsibilities	Collaborators
Check every quote against the segment text; resolve IDs and times from the database	TaskEvidence, Segment
Create tasks or revisions idempotently per class, stage, transcript and prompt version	Task, TaskRevision, ProcessingRun

CRC-18 · GoogleCalendarService

Service · planned (S3: J3.x)

Responsibilities	Collaborators
Create or update exactly one event per confirmed task version, with a deterministic event ID	Task, Google Calendar API
Keep a local cache of upcoming events for the dashboard and chat tools	CalendarEvent
Detect conflicts between timed events and same-day deadlines	Task
Mark the integration `disconnected` on `invalid_grant` and block syncs until reconnection	GoogleCredential

CRC-19 · NotificationScheduler

Service (ARQ cron) · planned (S4: J4.x)

Responsibilities	Collaborators
Compute reminder times per confirmed task and due version, in the student's local time	Task, User
Group and deduplicate reminders with a unique key; build daily and weekly digests	Notification
Send through the student's Gmail, opt-in only	Gmail API, GoogleCredential
Mark an uncertain send as `delivery_unknown` and never resend it automatically	Notification

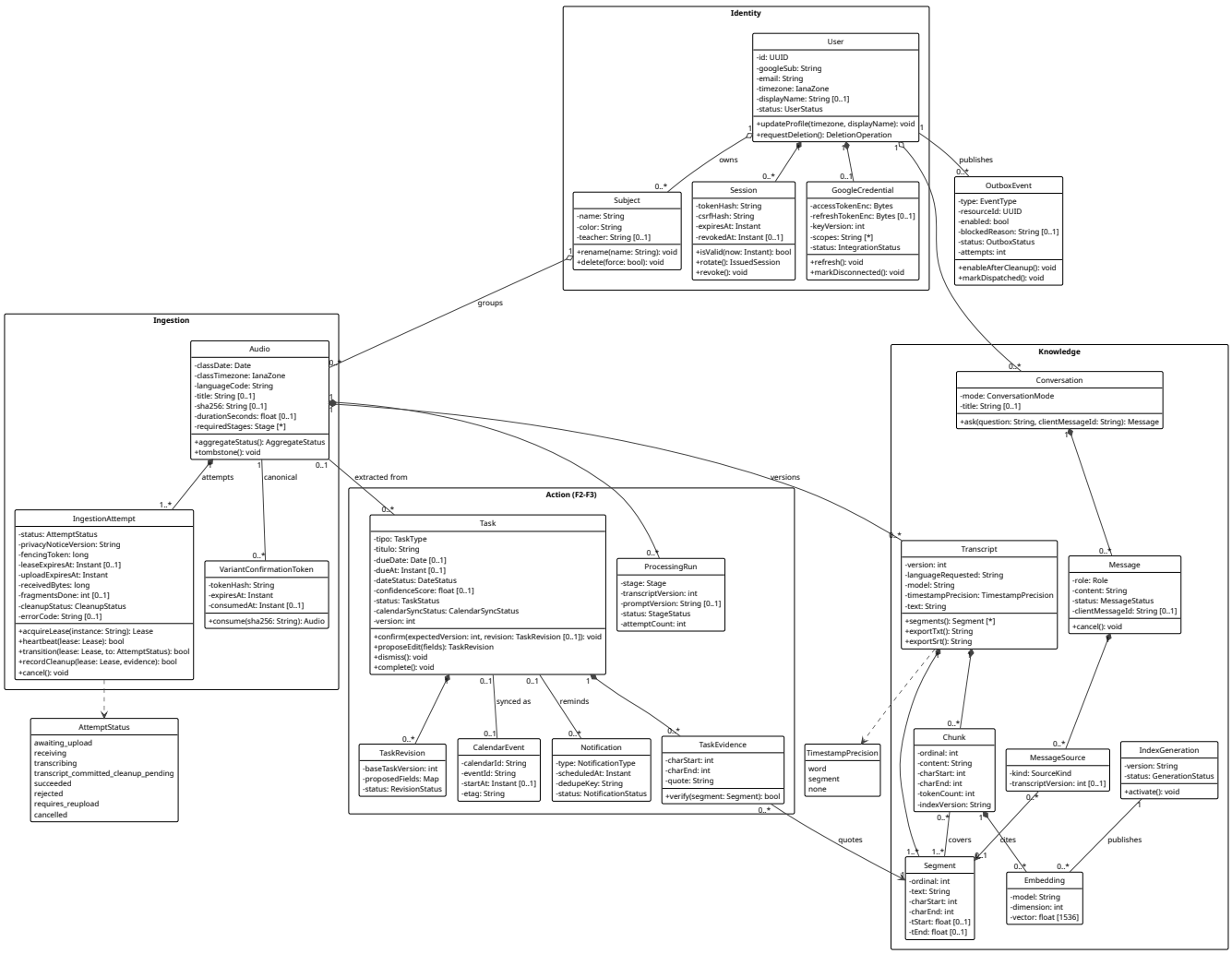
3. Class Diagram

The design is modelled with two UML class diagrams, following the notation summarized at uml-diagrams.org:

- **Figure 3.1** shows the persistent domain model: entities, their attributes with types and multiplicities, their domain operations, and the relationships between them.
- **Figure 3.2** shows the service layer: the services that run the business processes, the interfaces that isolate external providers, and the adapters that implement them.

3.1 Domain model

Figure 3.1 · Domain model (persistent entities)



Notation: `-` private attribute, `+` public operation, `[0..1]` optional, `[*]` collection.

Relationship	Kind	Meaning
User ◆— Session, GoogleCredential	Composition	A session or a Google credential cannot exist without its user; deleting the account deletes them
User ◆— Subject ◆— Audio	Aggregation	A subject groups classes; a class always belongs to exactly one subject of the same user
Audio ◆— IngestionAttempt	Composition, `1..*`	Every class has at least the attempt that created it; a re-upload adds another attempt
Audio ◆— Transcript ◆— Segment	Composition	Transcripts are versioned per class; segments are the unit of evidence
Chunk — Segment	Association, many-to-many	A chunk covers one or more segments; overlap between chunks is allowed (`chunk_segments`)
Chunk ◆— Embedding — IndexGeneration	Composition and association	One embedding per chunk and index generation; a generation is published atomically
Conversation ◆— Message ◆— MessageSource ◇ Segment	Composition and directed association	Each answer records every source delivered to the model, so a deletion can purge it
Audio — Task	Association `0..1` to `0..*`	Extracted tasks come from one class; manual tasks have none
Task ◆— TaskEvidence ◇ Segment	Composition and directed association	The quote and character span that justify a task
Task ◆— TaskRevision	Composition	Pending edits of a confirmed task (business rule BR-15)
Task — CalendarEvent, Notification	Association	At most one Calendar event per task; zero or more reminders
User — OutboxEvent	Association	Durable work published in the same transaction as its cause

Design decisions visible in the model:

- **No audio attribute anywhere.** `Audio` is the record of a class, not a file: it has no

storage key, path or blob (BR-01).

- **Tenant ownership.** Every entity belongs, directly or through its parent, to one `User`.

In the database this becomes the composite keys of section 4.

- **Optimistic concurrency and fencing.** `Task.version` protects confirmations;

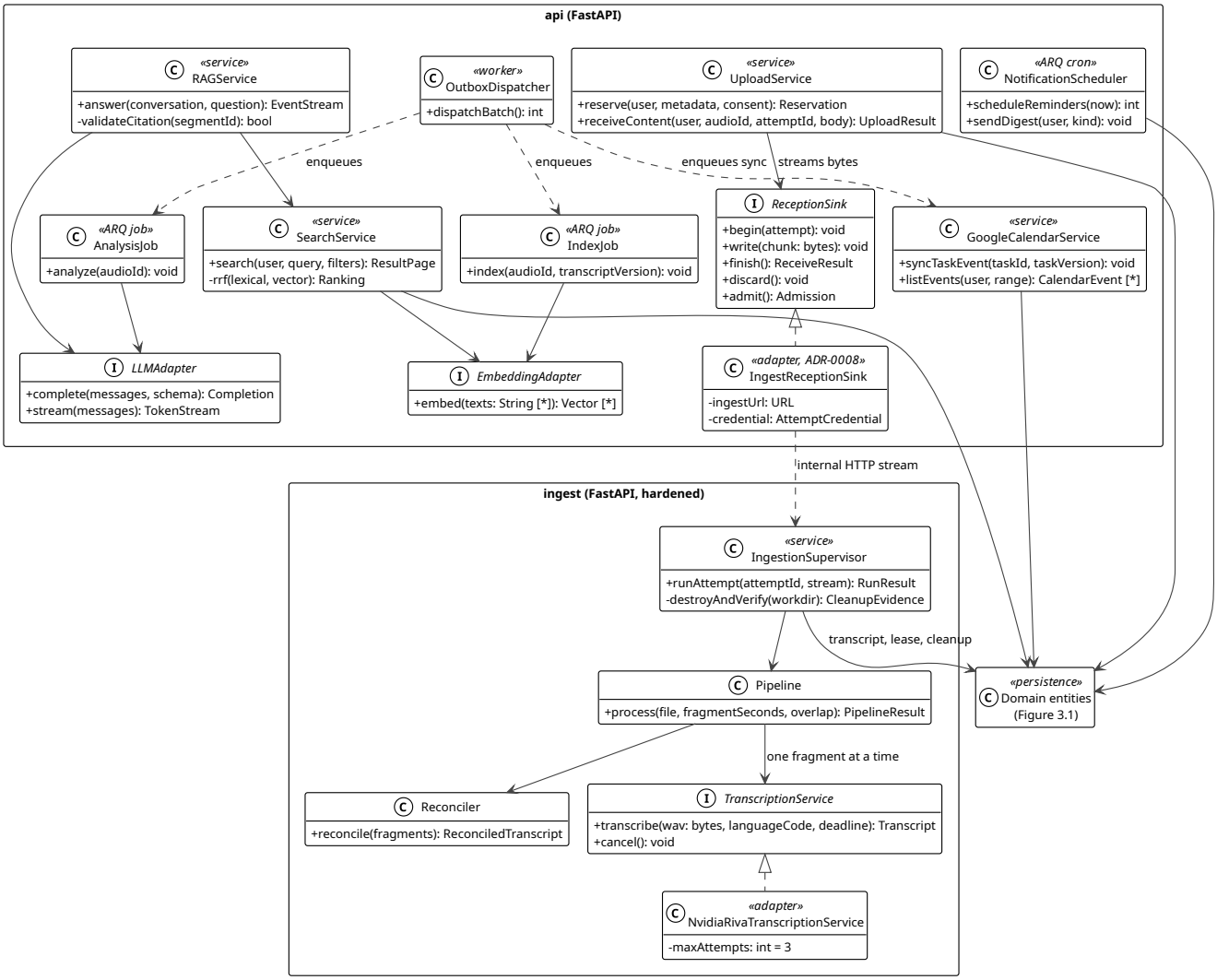
`IngestionAttempt.fencingToken` prevents an expired owner from publishing a transcript.

- **Status enumerations** are closed sets. `AttemptStatus` and `TimestampPrecision` are drawn as

examples; the others (`CleanupStatus`, `StageStatus`, `TaskStatus`, `CalendarSyncStatus`, `NotificationStatus`…) follow the state machines in DISENO_BD_API_SPRINTS.md §2.10.

3.2 Service layer

Figure 3.2 · Service layer: interfaces, adapters and dependencies



- **Ports and adapters.** `TranscriptionService`, `LLMAdapter`, `EmbeddingAdapter` and

`ReceptionSink` are interfaces. Today's adapters are `NvidiaRivaTranscriptionService` (NVIDIA Riva over gRPC) and `IngestReceptionSink` (the internal stream from the API to the ingestion service, ADR-0008). Changing a provider means a new adapter plus contract tests, never a change in the services (BR-18).

- **Two processes for audio and text.** Everything that touches audio lives in the hardened

`ingest` service; everything that works on text lives in `api` and its ARQ workers. Audio never crosses into Redis or the workers.

- **Asynchronous work goes through the outbox.** The `OutboxDispatcher` enqueues

`IndexJob`, `AnalysisJob` and Calendar syncs only after the event is enabled; this is what enforces "deletion before text work" (BR-02).

3.3 Mapping to code

Class	Location	Status
Domain entities of Identity, Ingestion and Knowledge	`apps/api/app/models/`	Implemented (migrations `0001`, `0002`)
Entities of the Action package	—	Planned (S3–S4)
`UploadService`, `ReceptionSink`	`apps/api/app/services/uploads.py`, `services/audios.py`	Implemented; `IngestReceptionSink` planned (A2.13)
`IngestionSupervisor`, `Pipeline`, `Reconciler`	`apps/ingest/maulwurf_ingest/supervisor.py`, `pipeline.py`, `audio/reconcile.py`	Implemented as a library (not reachable yet)
`TranscriptionService`, `NvidiaRivaTranscriptionService`	`apps/ingest/maulwurf_ingest/asr/riva_client.py`	Implemented, not wired (S2.10)
`OutboxDispatcher`, `IndexJob`, `AnalysisJob`	`apps/api/app/workers/`	Dispatcher implemented; jobs are stubs
`SearchService`, `RAGService`, `GoogleCalendarService`, `NotificationScheduler`, adapters	—	Planned (S2–S4)

The method names in the diagrams describe the design contract; where code already exists the real functions may be module-level (for example `services/uploads.py` exposes `receive_content`) rather than methods of a class.

4. Relational Database Model

The database is **PostgreSQL 16 with the `pgvector` extension**: one database holds business data, vectors and the outbox. The model has **27 tables**. 18 of them already exist in the Alembic migrations `0001_core_schema_s1` and `0002_s2_schema`; the other 9 belong to sprints S3–S4 and are designed but not migrated yet. The normative DDL, with every `CHECK` constraint and index, is in DISENO_BD_API_SPRINTS.md §2; the figures below are generated from that DDL, column by column.

4.1 Conventions

Convention	Rule
Primary keys	`uuid` generated with `gen_random_uuid()`; join tables use composite keys
Tenant ownership	Every business table has `user_id`. References between tenant tables are **composite foreign keys `(user_id, id)`**, so a row can never point to another user's row; each parent table therefore declares `UNIQUE (user_id, id)`
Deletion	Soft delete (`deleted_at` tombstone) on `users`, `subjects`, `audios` and `conversations`; derived rows are purged by cascades and purge jobs
Enumerations	`text` columns with a `CHECK (... IN (...))` list, so new values are added with a controlled migration
Time	`timestamptz` everywhere; `created_at` / `updated_at` default `now()`. Class dates are `date` with an explicit IANA timezone
No audio	No table has a storage key, path or blob for audio (BR-01); the outbox carries only IDs
Versioning	Transcripts by `(audio_id, version)`; chunks by `(transcript_id, index_version, ordinal)`; embeddings by `(chunk_id, index_version)`; runs by `(audio_id, stage, transcript_version, config_version)`

4.2 ER diagrams

Legend: `` marks a `NOT NULL` column; `«PK»` and `«FK»` mark key columns; white entities are implemented, grey entities with `«planned»` are not migrated yet. Crow's-foot notation: `||` exactly one, `|o` zero or one, `o{` zero or many. To keep the figures readable, the `user_id  users` edge is drawn only for the root tables; every other tenant table also has it.

Figure 4.1 · Overview of the relational model (27 tables)

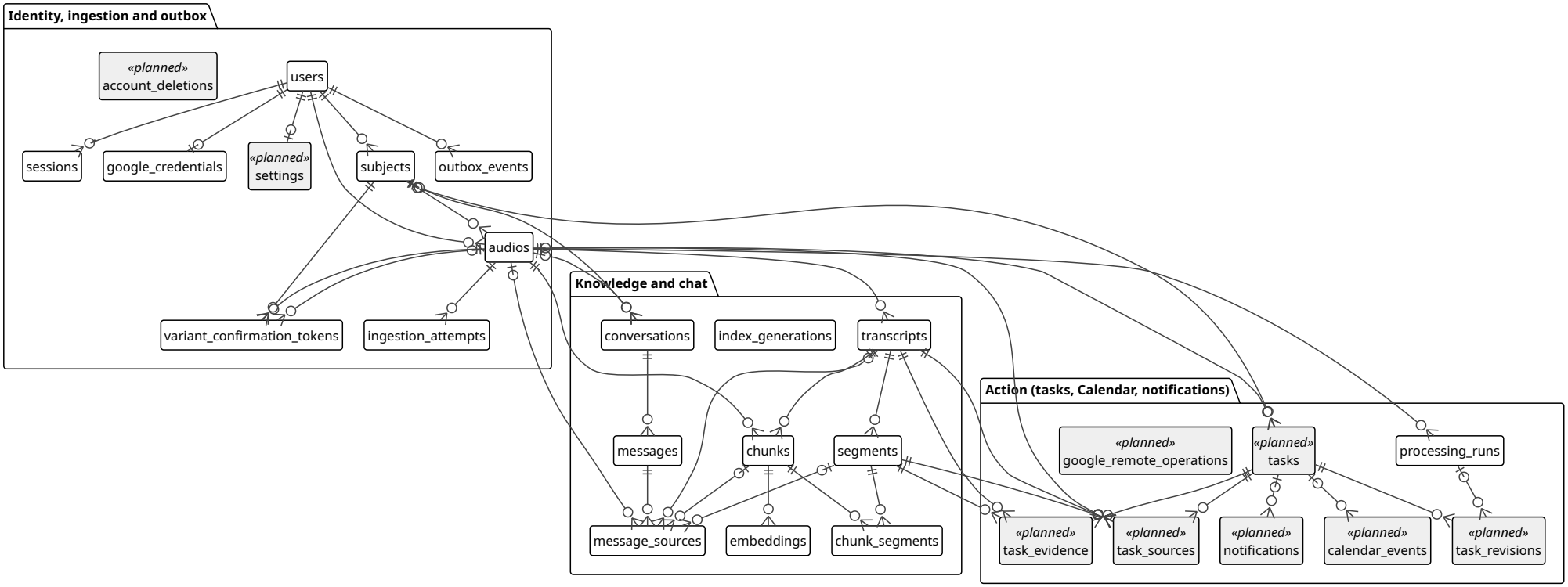


Figure 4.2 · Identity, ingestion and outbox

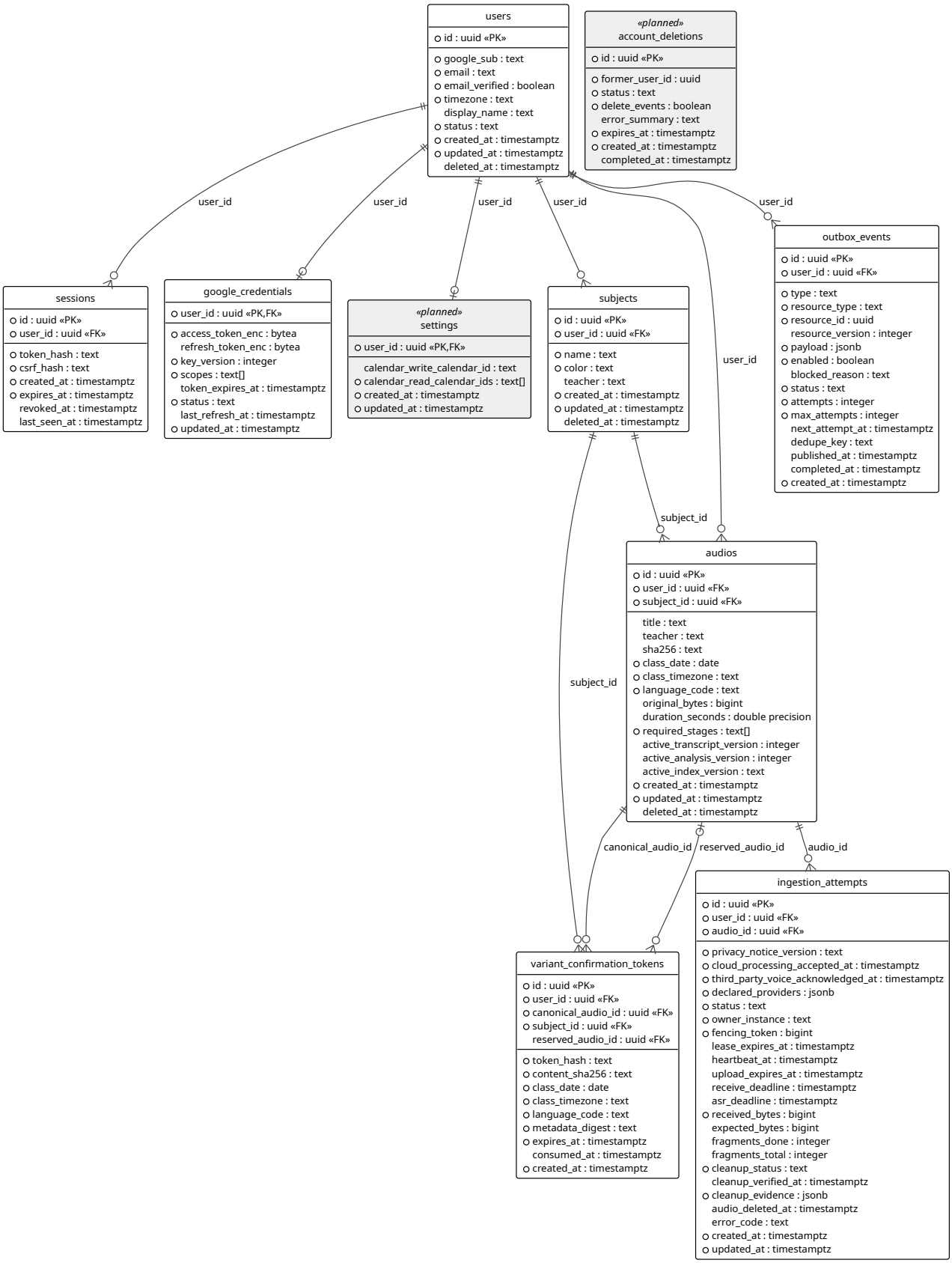


Figure 4.3 · Transcripts, index and chat

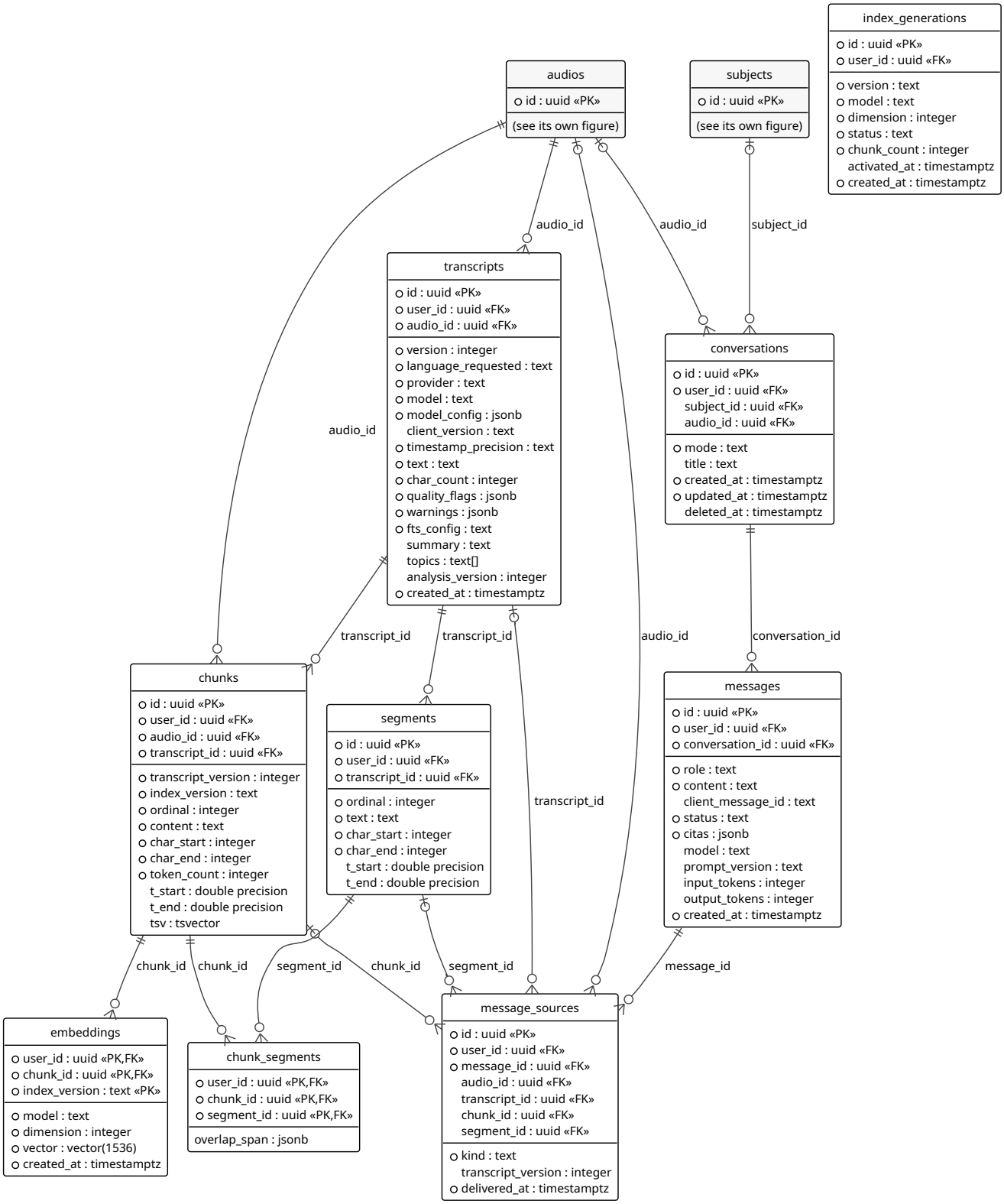
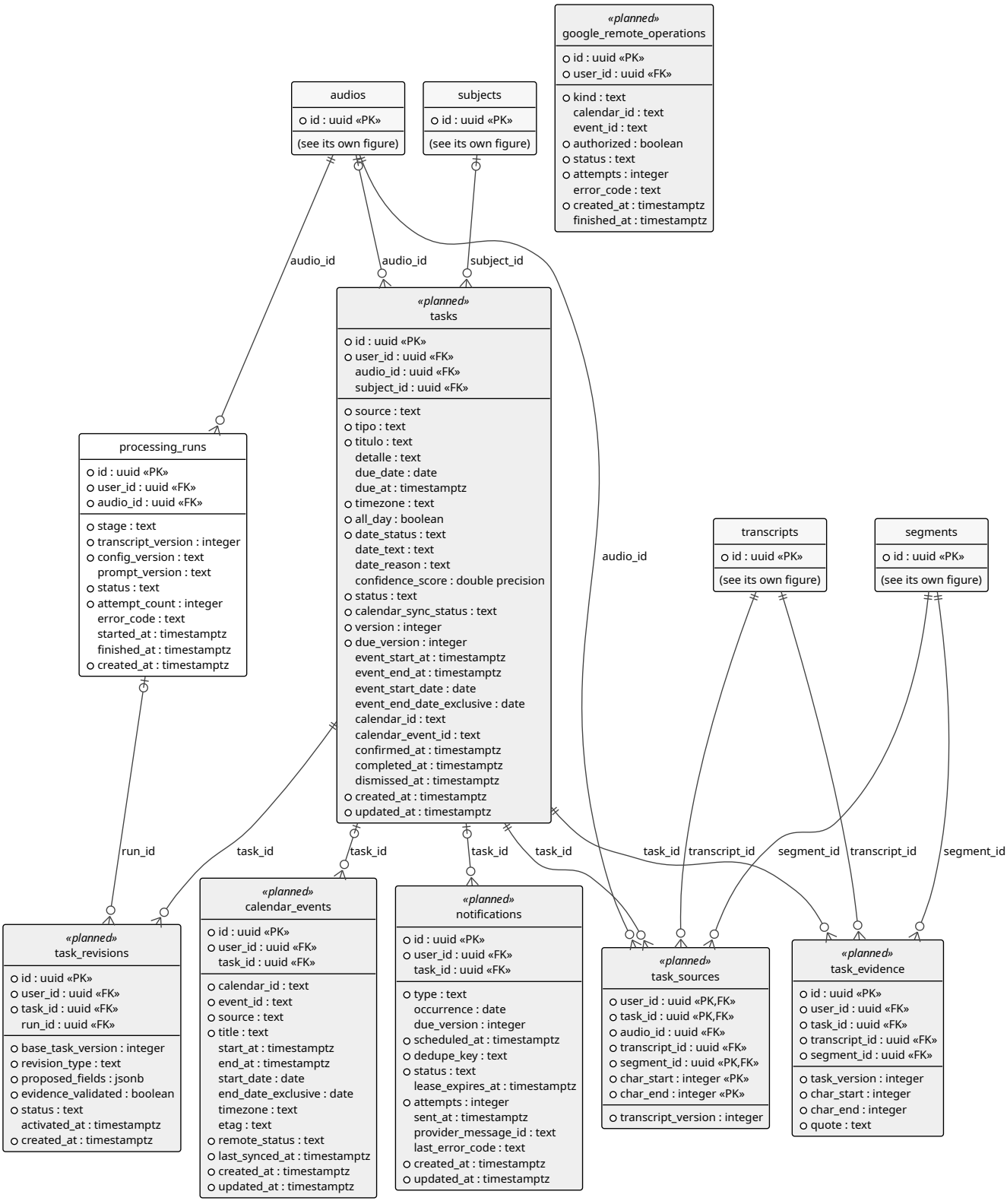


Figure 4.4 · Tasks, Calendar and notifications



4.3 Table definitions

Foreign keys written as `(user_id, x)` are composite tenant keys.

Table	Primary key	Foreign keys → table	Purpose	Status
users	id	—	Student identity (Google sub), profile and account status	Implemented (0001)
sessions	id	user_id → users	Opaque sign-in sessions; only token and CSRF hashes	Implemented (0001)
google_credentials	user_id	user_id → users	Encrypted Google tokens, scopes and key version	Implemented (0001)
settings	user_id	user_id → users	Calendar preferences (write and read calendars)	Planned
account_deletions	id	—	Residual record of an account deletion (no personal data)	Planned
subjects	id	user_id → users	Courses that group classes	Implemented (0001)
audios	id	user_id → users; (user_id, subject_id) → subjects	Class record: metadata and content identity, never the audio	Implemented (0001)
ingestion_attempts	id	user_id → users; (user_id, audio_id) → audios	Consent, lease and fencing, ASR state, progress and cleanup	Implemented (0001)
variant_confirmation_tokens	id	user_id → users; (user_id, canonical_audio_id) → audios; (user_id, subject_id) → subjects; (user_id, reserved_audio_id) → audios	Single-use confirmation to upload the same audio with another context	Implemented (0002)
outbox_events	id	user_id → users	Durable work items with IDs only; cleanup gate for index and analysis	Implemented (0001)
transcripts	id	user_id → users; (user_id, audio_id) → audios	Versioned transcript text, model and timestamp precision	Implemented (0002)

Table	Primary key	Foreign keys → table	Purpose	Status
segments	id	(user_id, transcript_id) → transcripts	Ordered transcript segments with character offsets and optional times	Implemented (0002)
chunks	id	(user_id, audio_id) → audios; (user_id, transcript_id) → transcripts	Indexing units with offsets, token count and full-text vector	Implemented (0002)
chunk_segments	user_id, chunk_id, segment_id	(user_id, chunk_id) → chunks; (user_id, segment_id) → segments	Which segments each chunk covers (evidence)	Implemented (0002)
embeddings	user_id, chunk_id, index_version	(user_id, chunk_id) → chunks	One vector per chunk and index generation	Implemented (0002)
index_generations	id	user_id → users	Index versions per user; one active at a time	Implemented (0002)
conversations	id	user_id → users; (user_id, subject_id) → subjects; (user_id, audio_id) → audios	Chat scope: all classes, a subject or a class	Implemented (0002)
messages	id	(user_id, conversation_id) → conversations	Chat history with status, citations and token usage	Implemented (0002)
message_sources	id	(user_id, message_id) → messages; (user_id, audio_id) → audios; (user_id, transcript_id) → transcripts; (user_id, chunk_id) → chunks; (user_id, segment_id) → segments	Every source delivered to the model, for purging	Implemented (0002)
processing_runs	id	user_id → users; (user_id, audio_id) → audios	Idempotent index and analysis runs per transcript and config version	Implemented (0002)
tasks	id	user_id → users; (user_id, audio_id) → audios; (user_id, subject_id) → subjects	Extracted or manual tasks with dates, status and Calendar sync	Planned

Table	Primary key	Foreign keys → table	Purpose	Status
task_evidence	id	(user_id, task_id) → tasks; (user_id, transcript_id) → transcripts; (user_id, segment_id) → segments	Literal quote and span that justify a task	Planned
task_sources	user_id, task_id, segment_id, char_start, char_end	(user_id, task_id) → tasks; (user_id, audio_id) → audios; (user_id, transcript_id) → transcripts; (user_id, segment_id) → segments	Segments a task was derived from	Planned
task_revisions	id	(user_id, task_id) → tasks; (user_id, run_id) → processing_runs	Pending edits and re-analysis results for a task	Planned
calendar_events	id	user_id → users; (user_id, task_id) → tasks	Local cache of Google Calendar events	Planned
google_remote_operations	id	user_id → users	Traceable remote Google operations (for example event deletion)	Planned
notifications	id	user_id → users; (user_id, task_id) → tasks	Scheduled reminders and digests with delivery status	Planned

4.4 Key constraints and indexes

Constraint or index	Table	Purpose
audios_dedupe_identity (unique, partial)	audios	Deduplication identity: user, SHA-256, language, subject, class date and timezone (BR-07)
attempts_one_active (unique, partial)	ingestion_attempts	At most one active attempt per class
ck_outbox_events_cleanup_gate	outbox_events	index_requested and analyze_requested are born disabled with blocked_reason = 'cleanup_pending' (BR-02)
UNIQUE NULLS NOT DISTINCT (type, resource_id, resource_version, dedupe_key)	outbox_events	Idempotent publication
ck_transcripts_char_count_matches_text and timestamp_precision IN ('word','segment','none')	transcripts	Consistent text length; honest timestamp precision (BR-09)
embeddings_hnsw (HNSW, cosine) and chunks_fts (GIN)	embeddings, chunks	Vector and full-text branches of hybrid search
index_generations_one_active (unique, partial)	index_generations	A single active index generation per user

Constraint or index	Table	Purpose
`messages_client_idem` (unique, partial)	`messages`	Idempotent sending through `client_message_id`
`due_date` / `due_at` mutually exclusive, `confidence_score` in [0, 1]	`tasks`	Date rules of BR-14

Row-level security policies on every tenant table are the planned second barrier of ADR-0006 (ticket A2.17); today isolation is enforced by application filtering and the composite foreign keys.

5. Mockups

The mockups are low-fidelity wireframes. They describe the primary screens and the information architecture, not final visual styling. They use the current product vocabulary and preserve the privacy and human-confirmation rules from section 1.

5.1 Screen inventory

Screen	Route	User goal	Main states represented
Sign in	`/login`	Start or resume a session with Google	OAuth error and successful sign-in
Dashboard	`/`	See agenda, review work and ingestion status at a glance	ready, processing, partial, failed
Subjects	`/materias`	Create, edit and delete course subjects	active classes block deletion
Upload class	`/biblioteca/upload`	Provide metadata, consent and the recording before bytes move	language required, provisional limits
Library	`/biblioteca`	Browse classes and distinguish retry from upload again	per-stage status and real progress
Transcript reader	`/biblioteca/{audio_id}`	Read and export the durable transcript	segments, timestamps, summary and deletion
Chat	`/chat`	Ask across notes with validated citations	streaming, citations and abstention
Review inbox	`/inbox`	Review extracted tasks before Calendar writes	missing/ambiguous date and confirmation
Settings	`/settings`	Manage profile, integrations, notifications and deletion	disconnected integration and deletion option

5.2 Wireframes

The sources are editable PlantUML Salt diagrams. Their SVGs are embedded in the PDF and can be regenerated with `python3 Workshop-2/build.py diagrams`.

Sign in

Maulwurf . . [Sign in]

Your classes, turned into study actions

Upload a class recording. Maulwurf keeps only the text, answers with citations and proposes tasks you confirm before anything reaches your calendar.

1. Sign in with your Google account
2. Upload the recording of a class and choose its language
3. Search, ask and review proposed tasks

Continue with Google

Audio is deleted after transcription and is never stored. Calendar and Gmail are connected later, only if you choose to.

Google sign-in expired; please try again.

The screen does not request Google Calendar or Gmail scopes. Those permissions are incremental, optional and requested only from the integrations settings.

Dashboard

Maulwurf Dashboard Library Subjects Chat Inbox Settings maria@unal.edu.co

Dashboard

+ Upload a class

Upcoming due dates

Mon 21 · Calculus II · Midterm exam (all day) synced
Wed 23 · Physics I · Lab report 3 · 23:59 pending sync
Fri 25 · History · Read chapter 4 (all day) not in Calendar

To confirm (3)

Exam · Calculus II · "the midterm will be on Monday..."
Reading · History · date missing

Open review inbox

Ingestion status

Class 03 · Calculus II transcribing · 12/40 fragments
Class 02 · Physics I ready
Class 01 · History upload again (audio lost)

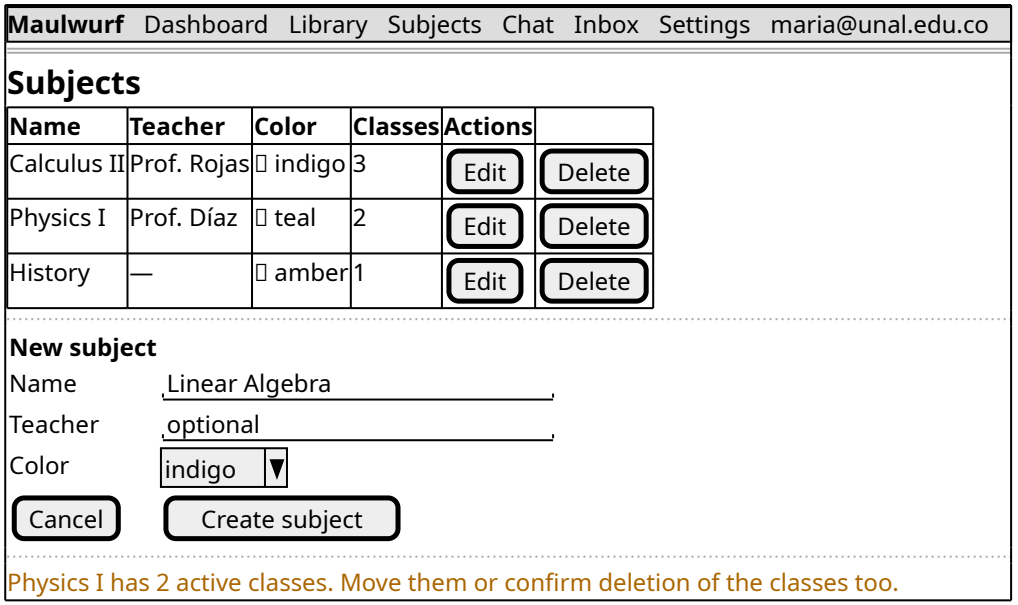
This week (Google Calendar + Maulwurf)

Mon	Tue	Wed	Thu	Fri
Exam		Lab 3	Lecture	Reading
	Seminar			

Calendar connected · last sync 5 min ago

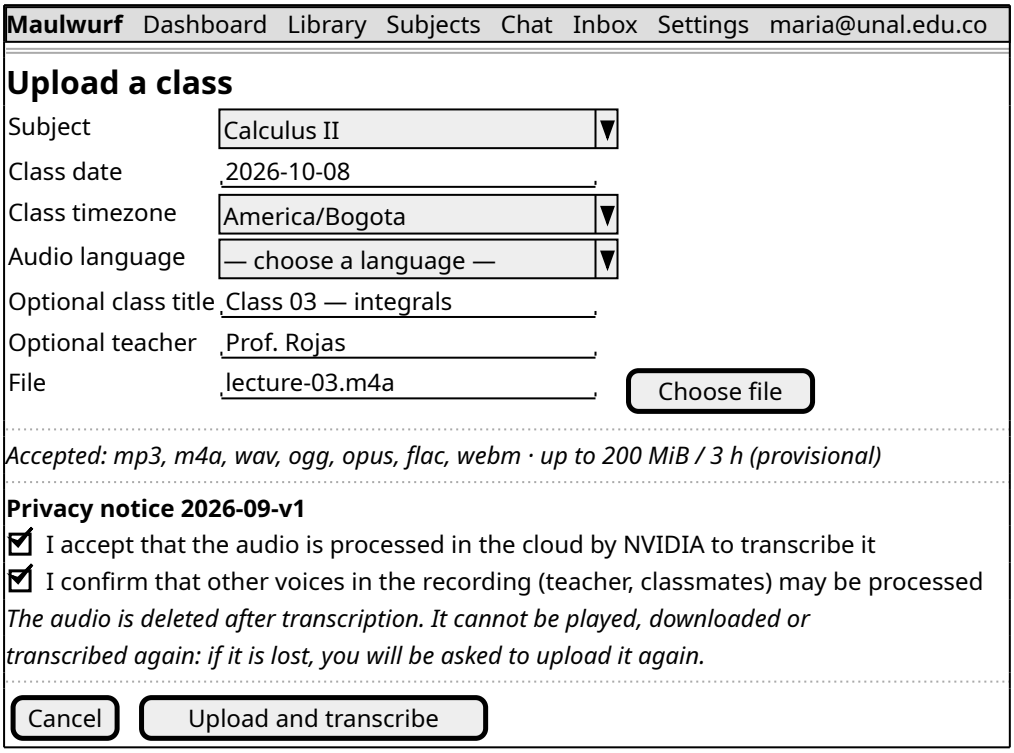
The dashboard deliberately shows stage status instead of an invented percentage. It combines local Maulwurf tasks with the Calendar cache and explains when an integration is stale or disconnected.

Subjects


Maulwurf Dashboard Library Subjects Chat Inbox Settings maria@unal.edu.co
Subjects
Name Teacher Color Classes Actions
Calculus II Prof. Rojas indigo 3 Edit Delete
Physics I Prof. Díaz teal 2 Edit Delete
History — amber 1 Edit Delete
New subject
Name Linear Algebra
Teacher optional
Color indigo
Cancel Create subject
Physics I has 2 active classes. Move them or confirm deletion of the classes too.

Deletion is guarded when a subject has active classes. The user must move the classes or explicitly confirm the destructive operation, as required by M2.

Upload class



The form requires subject, class date, timezone, language and two acknowledgements before the `PUT` begins. It shows the effective limits from `/ingestion/capabilities`, while marking the 200 MiB / 3 h values as provisional until D4 is closed.

Library

Maulwurf Dashboard Library Subjects Chat Inbox Settings maria@unal.edu.co

Library

All subjects ▼ Search classes… (Ctrl+K)

Class 03 — integrals · Calculus II · 2026-10-08 · Spanish

Uploading… 84.2 of 96.0 MiB received

[═]

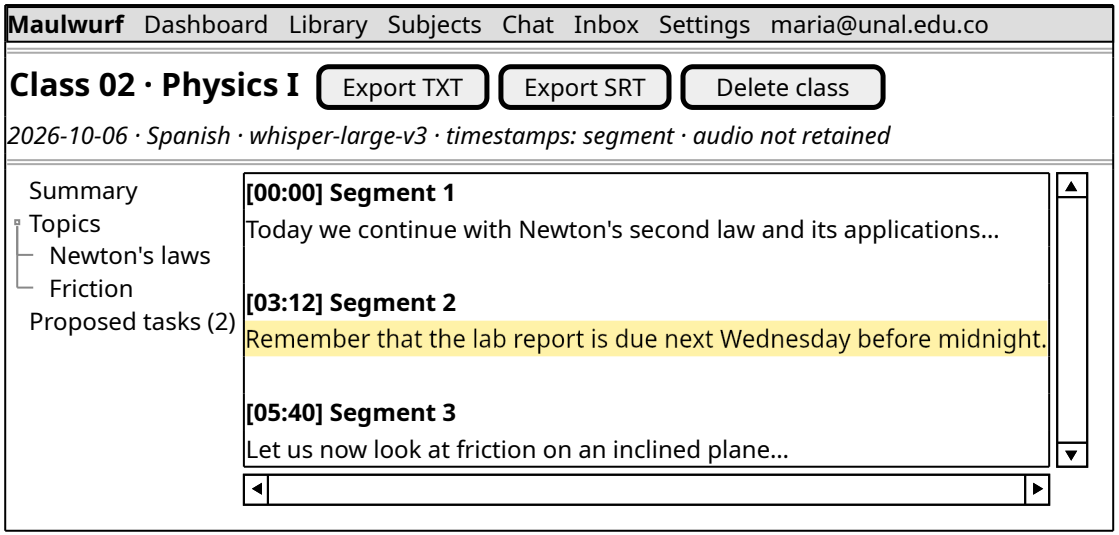
Cancel upload

Class	Subject	Date	Transcript	Index	Analysis	Status
Class 02	Physics I	10-06	succeeded	succeeded	succeeded	ready
Class 01	Calculus II	10-01	succeeded	failed	succeeded	partial · [Retry index]
Class 01	History	09-29	audio lost	—	—	failed · [Upload again]

Audio is not retained: only transcripts, indexes and tasks are stored.

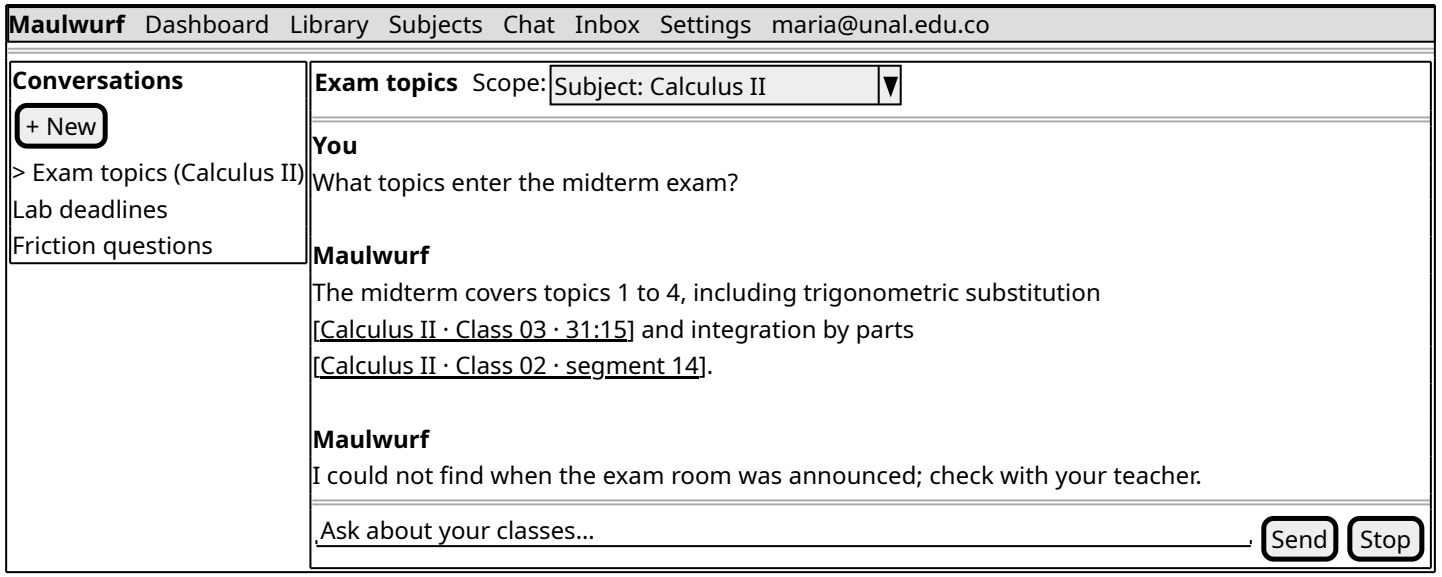
`Retry` is available for text-processing stages after a usable transcript exists. `Upload again` is used when audio or the pre-commit temporary state was lost; the two actions must not be merged.

Transcript reader


Maulwurf Dashboard Library Subjects Chat Inbox Settings maria@unal.edu.co
Class 02 · Physics I
Export TXT
Export SRT
Delete class
2026-10-06 · Spanish · whisper-large-v3 · timestamps: segment · audio not retained
Summary
Topics
Newton's laws
Friction
Proposed tasks (2)
[00:00] Segment 1
Today we continue with Newton's second law and its applications...
[03:12] Segment 2
Remember that the lab report is due next Wednesday before midnight.
[05:40] Segment 3
Let us now look at friction on an inclined plane...

The reader never offers audio playback or download. SRT is visible only when timestamps are valid. Segment links are stable evidence anchors, and the permanent notice says that audio was not retained.

Chat


Maulwurf Dashboard Library Subjects Chat Inbox Settings maria@unal.edu.co
Conversations
+ New
> Exam topics (Calculus II)
Lab deadlines
Friction questions
Exam topics Scope: Subject: Calculus II
You
What topics enter the midterm exam?
Maulwurf
The midterm covers topics 1 to 4, including trigonometric substitution
[Calculus II · Class 03 · 31:15] and integration by parts
[Calculus II · Class 02 · segment 14].
Maulwurf
I could not find when the exam room was announced; check with your teacher.
Ask about your classes...
Send
Stop

The chat uses a scope selector and renders only database-validated citations. If the evidence is insufficient, it says so instead of inventing a date, source or timestamp.

Review inbox

Maulwurf Dashboard Library Subjects Chat Inbox Settings maria@unal.edu.co

Review inbox · 3 proposals

Exam · Calculus II · from Class 03

Task title: Midterm exam — topics 1 to 4

Due: (•) All day ( ) At a time

Date: 2026-10-21

Timezone: America/Bogota

"the midterm exam will be on Monday, September 21…" · [View segment]

Confidence signal 0.92 (not a guarantee)

Dismiss | Confirm and add to Calendar

Reading · History · from Class 01 · **needs review: date missing**

Task title: Read chapter 4

Due: ( ) All day ( ) At a time

Date: pick a date

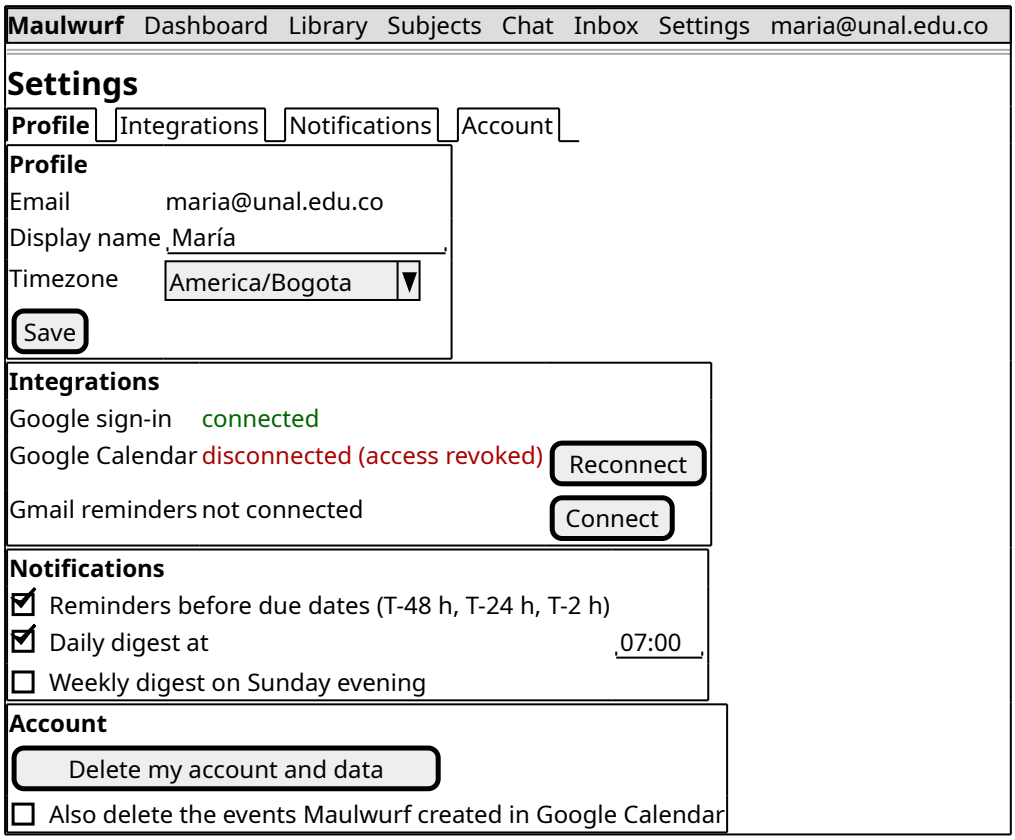
"for next week read chapter four" · [View segment]

Dismiss [Confirm]

Nothing is written to Google Calendar until you confirm.

A task proposal shows its literal quote, source segment and confidence signal. The signal is not a probability and never schedules an event by itself. The Calendar write happens only when the student presses **Confirm and add to Calendar**.

Settings



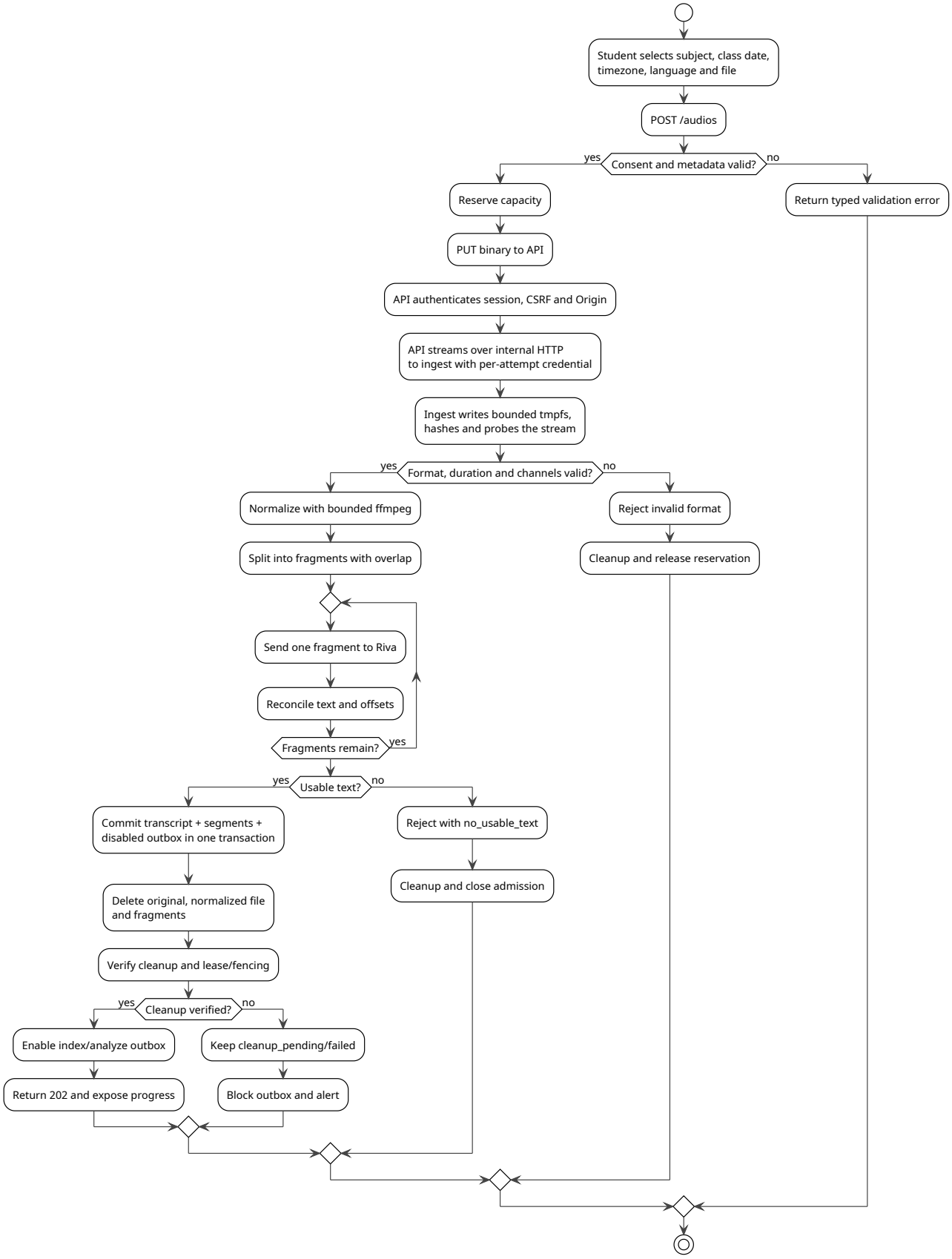
The settings screen separates profile, integrations, notifications and account deletion. Deleting the account has a separate choice for authorized deletion of events previously created by Maulwurf.

6. Business Model Processes

The process models use UML activity diagrams. The central process is the upload-to-transcript vertical slice because it crosses the most important trust boundaries: browser API ingestion service NVIDIA Riva durable text. The other two diagrams show the two irreversible business flows that require explicit controls: Calendar confirmation and account deletion.

6.1 Upload, transcription and verified cleanup

Figure 6.1 · Upload, transcription and cleanup

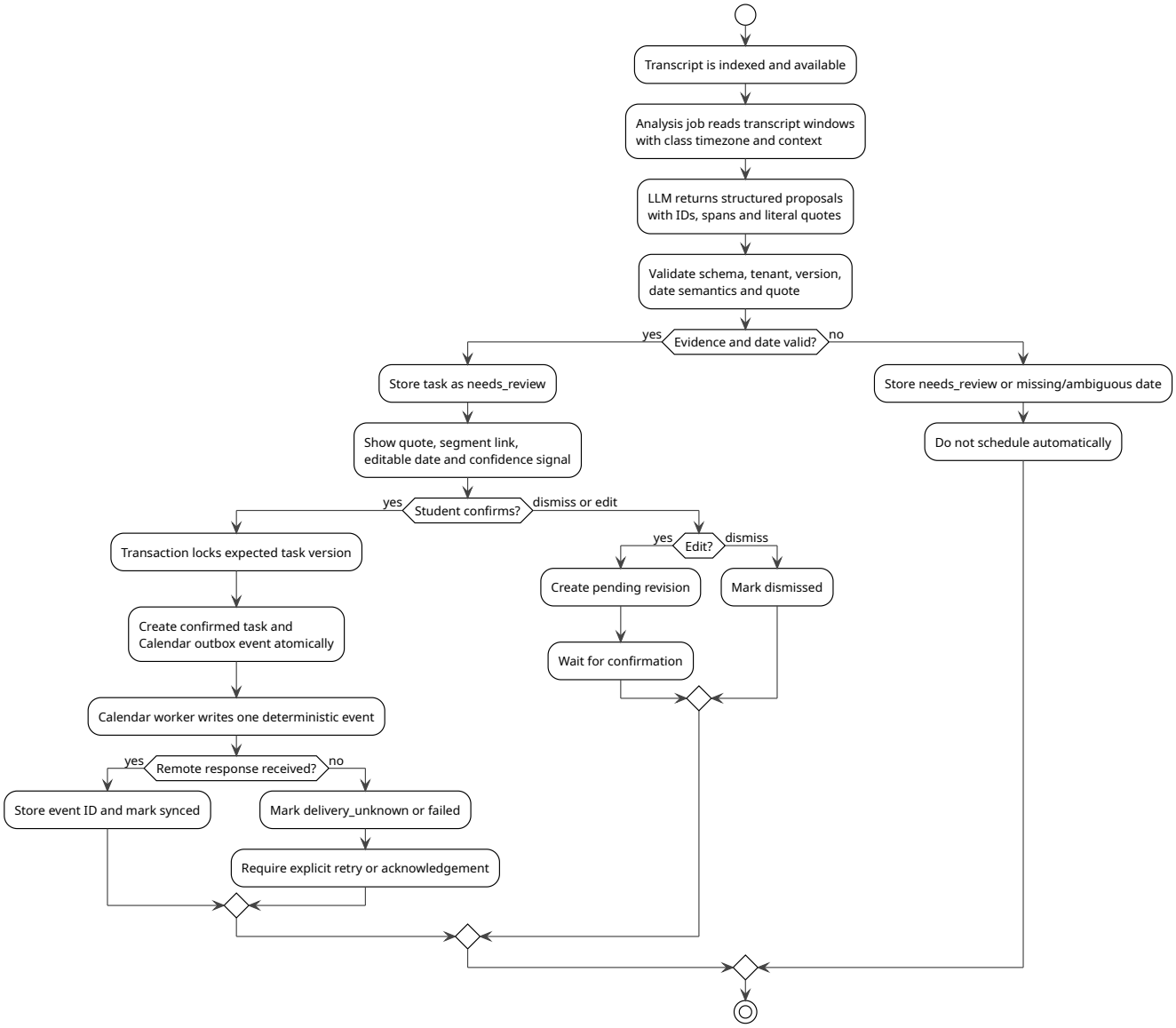


Inputs: authenticated session, subject, class date, class timezone, selected language, consent version and an audio stream. **Output:** a usable transcript and segments, or a typed terminal status. The

process has no branch that stores audio durably. `cleanup_pending` is not a success state: the outbox remains blocked until a supervisor verifies the temporary files and processes are gone.

6.2 Evidence-backed task to Calendar

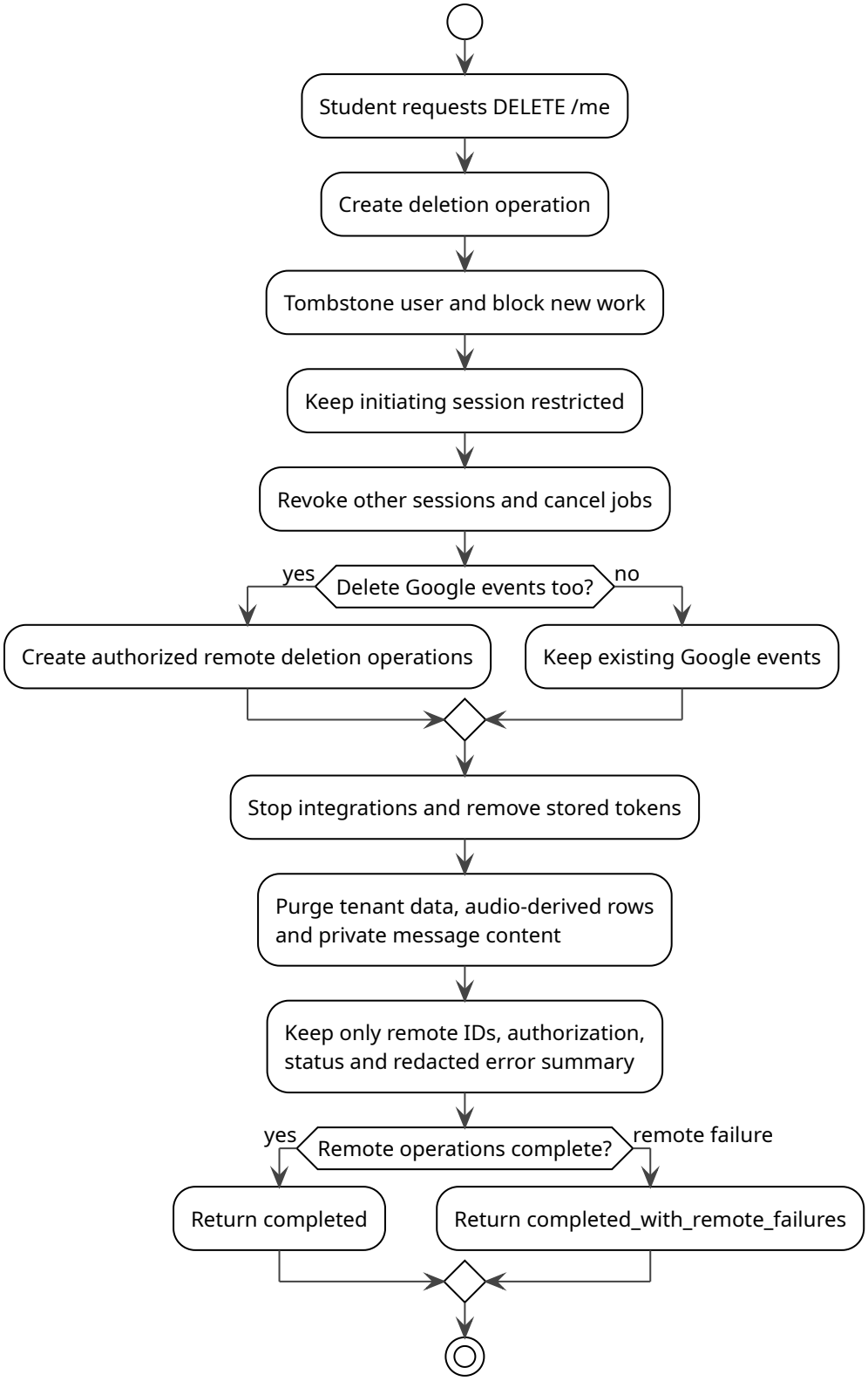
Figure 6.2 · Evidence-backed task to Calendar



The analysis worker is allowed to propose, not to schedule. The quote and source segment must be validated against the immutable transcript before the proposal enters the review inbox. A stale optimistic-locking version cannot confirm a task. A lost remote response is not retried blindly, because Google Calendar does not provide exactly-once delivery; the operation becomes `delivery_unknown` and requires an explicit decision.

6.3 Account deletion and retention boundary

Figure 6.3 · Account deletion and retention boundary



Deletion is asynchronous (`202`) because it coordinates local tombstones, sessions, jobs, tokens, derived data and optional authorized Google event deletion. The initiating session is restricted while

the operation runs. The residual operation contains only remote IDs, authorization, status and a redacted error; it must not preserve the student's transcript or other private content.

6.4 Process controls and exception paths

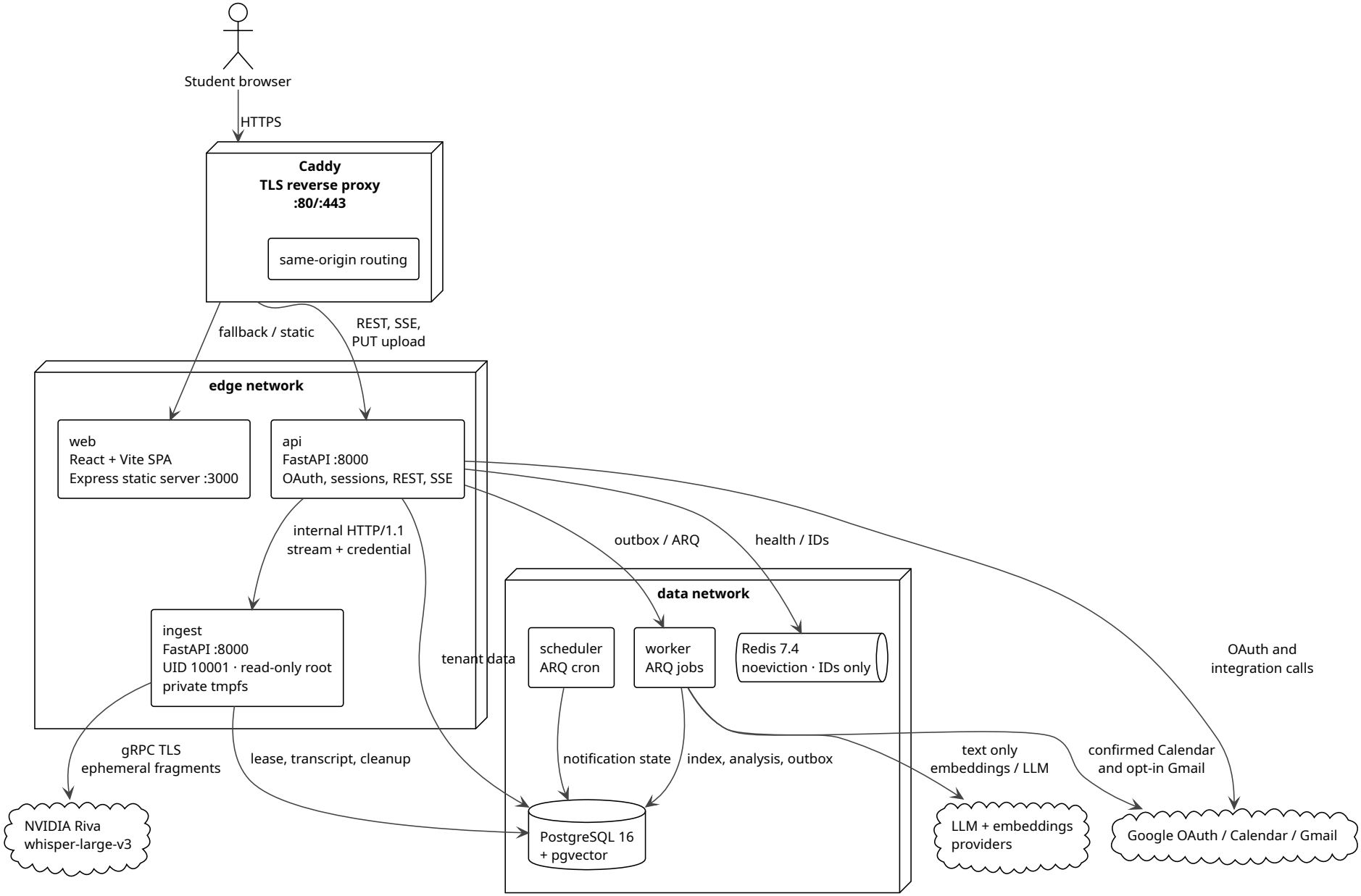
Process control	Normal path	Exception path	Business result
Upload admission	Consent and metadata validated before bytes	stale notice, invalid language, expired attempt	typed 4xx; no capacity or audio accepted
Reception	API streams with backpressure to private ingest tmpfs	disconnect, size limit, internal hop timeout	cancel/discard and verify cleanup
ASR	Riva returns text or valid offsets	transient retry, provider rejection, deadline	bounded retry; `rejected` or `requires_reupload`
Persistence	Transcript, segments and blocked outbox commit together	database failure	rollback; no partial transcript published
Cleanup	all temporary files and processes verified absent	cleanup check fails or host disappears	`cleanup_pending/failed`, outbox blocked, alert
Task extraction	validated JSON and source span	ambiguous date or invalid quote	`needs_review`; never Calendar write
Calendar	one confirmed task version creates one deterministic event	lost response or `invalid_grant`	`delivery_unknown` / `blocked`; explicit retry or reconnect
Account deletion	tombstone, revoke, purge and remote operations	remote deletion failure	`completed_with_remote_failures`, redacted residual record

7. Architecture Diagram

The architecture follows the accepted decisions in ADR-0007 (Vite + React SPA) and ADR-0008 (the API terminates the public `PUT` and streams to the internal ingestion service).

7.1 Component and deployment view

Figure 7.1 · Maulwurf software architecture



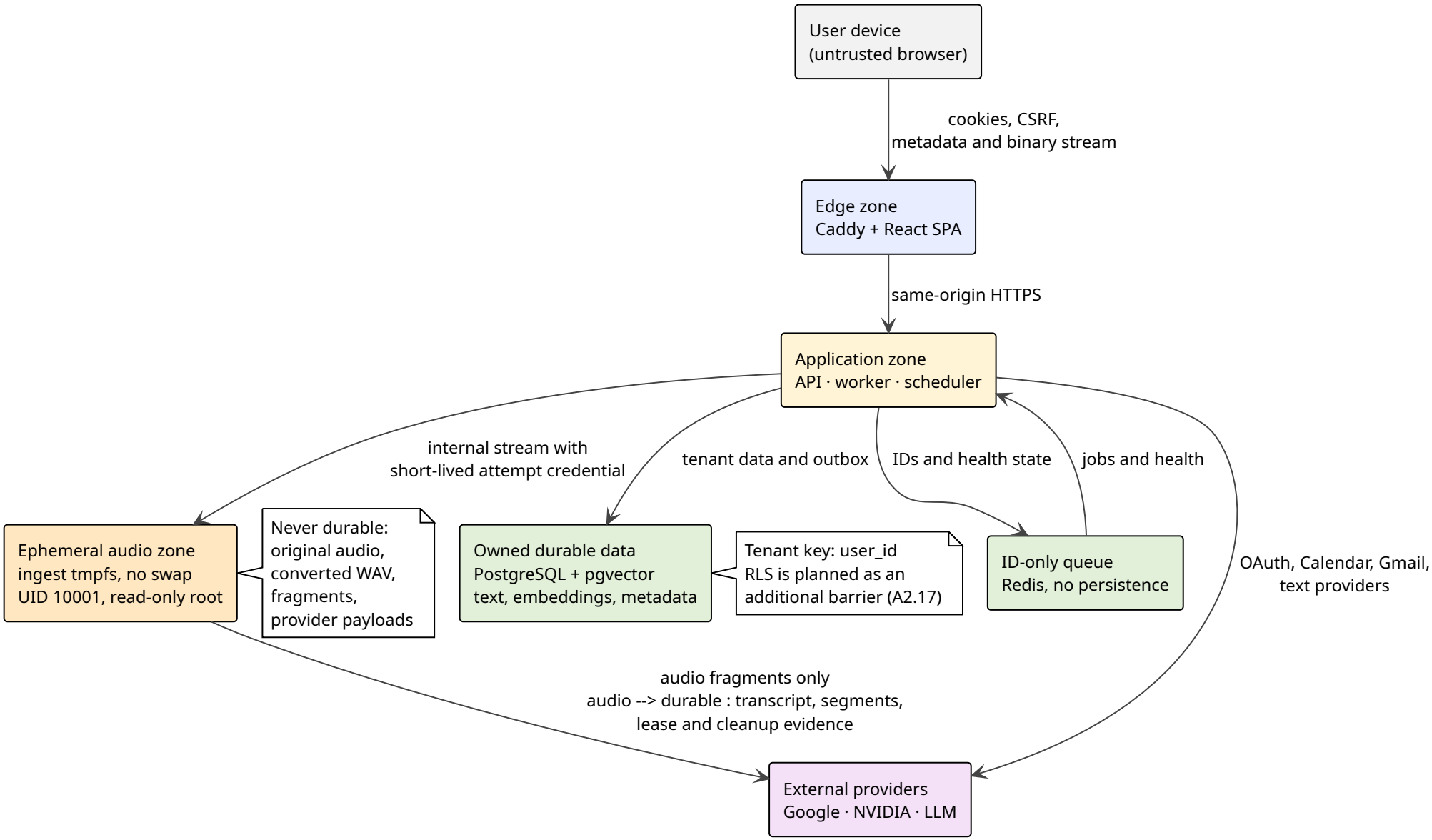
Component	Technology / runtime	Responsibility	Network and data boundary
Caddy	Caddy 2.8, local TLS	Same-origin reverse proxy and no-buffering upload route	Edge; public entry point
Web	React 19 + Vite 7 + TypeScript + Express 5	Static SPA and client-side routes	Edge; no secrets and no server-side private data
API	FastAPI + Pydantic v2 + SQLAlchemy async	OAuth, sessions, CSRF, metadata, public REST/SSE and public binary `PUT`	Edge + data; authenticates all browser mutations
Ingest	FastAPI, ffmpeg/ffprobe, Riva adapter	Internal streamed reception, bounded tmpfs, ASR, cleanup and fencing	Edge + data; UID 10001; never receives browser cookies
PostgreSQL	PostgreSQL 16 + pgvector	Tenant metadata, text, segments, vectors, chat and outbox	Data only; no durable audio
Redis	Redis 7.4, `noeviction`, no persistence	IDs, ARQ queue and health state	Data only; never audio or provider payloads
Worker	ARQ	Indexing, analysis, Calendar and notification jobs	Data only; works from text and IDs
Scheduler	ARQ cron	Bounded reminder and digest scheduling	Data only; one active scheduler role

Data flow

1. The browser connects only to Caddy over same-origin HTTPS. 2. Caddy serves the SPA and sends REST/SSE and the binary `PUT` to the API. 3. The API owns session, CSRF, Origin, tenant and pre-byte checks. It streams the binary body over internal HTTP/1.1 to `ingest` using the per-attempt credential defined by ADR-0008. 4. `ingest` holds audio only in the attempt's bounded tmpfs, sends fragments to Riva, commits text and cleanup evidence, and never puts audio in Redis or an ARQ job. 5. Workers read durable text and IDs from PostgreSQL/outbox. They call LLM, embeddings, Calendar and Gmail adapters only according to the confirmation and opt-in rules.

7.2 Trust zones

Figure 7.2 · Trust zones and data flow



The browser and external providers are outside the application's trust boundary. The API is the single identity authority. Ingestion is isolated because it handles audio and provider calls; it accepts only a short-lived internal credential and a fencing token. The durable boundary contains text and metadata, not audio. PostgreSQL tenant isolation is application-enforced today; `FORCE ROW LEVEL SECURITY` and `SET LOCAL app.user_id` are planned in A2.17.

7.3 Quality and security properties

- **Privacy:** no audio storage key, volume, blob, Redis payload, log, trace or backup is allowed.
- **Backpressure:** each hop streams; `ingest` processes one fragment at a time and rejects over-limit

input before unbounded memory growth.

- **Failure recovery:** lease, fencing token, cleanup evidence and the outbox gate prevent stale

workers from publishing text work.

- **Tenant isolation:** composite foreign keys, service-level ownership filters and the planned RLS

barrier keep users from crossing into one another's data.

- **Provider isolation:** Riva, LLM, embeddings, Google Calendar and Gmail sit behind adapters and

contract tests; external side effects are bounded and idempotent where the provider permits it.

Built 2026-10-13 from commit c3760e3.